

INFORME TÉCNICO DE ARQUITECTURA

Contenedorización con Docker vs Despliegue Nativo (JobFlow)

Trabajo Integrador Final | UTN FR Avellaneda | Tecnicatura Universitaria en Programación

Autor: Santiago Ezequiel Chavez | Septiembre 2026

1 ¿Por qué se incorporó Docker en el E-Commerce Gamer?

El proyecto E-Commerce Gamer es una solución Fullstack heterogénea (políglota): combina un backend en Java 21 con Spring Boot 3, un frontend SPA en React 18 / Vite, un servidor web Nginx y persistencia NoSQL en MongoDB Atlas.

El Propósito Principal de Docker en este Proyecto

Garantizar que el sistema completo pueda compilarse, configurarse y ejecutarse de forma 100% idéntica en cualquier computadora o servidor (Windows, Linux, macOS o Render en la nube) con un único comando, eliminando toda dependencia manual de JDK, Node.js o Maven en el host.

Conceptos Fundamentales aplicados en el E-Commerce:

Elemento	Función en el E-Commerce	Beneficio Técnico
backend/Dockerfile	Compila con Maven y ejecuta el JAR en JRE 21 Alpine.	Multi-stage: Reduce el peso de 600MB a 150MB y añade seguridad no-root.
frontend/Dockerfile	Compila React con Vite y lo monta en Nginx Alpine.	Multi-stage: No requiere Node en ejecución; Nginx sirve estáticos a alta velocidad.
frontend/nginx.conf	Servidor web HTTP con regla try_files \$uri /index.html.	Elimina los errores 404 al recargar rutas de React Router (SPA).
docker-compose.yml	Orquestador multi-contenedor con red interna y puertos.	Levanta Backend (8080) y Frontend (5176) sincronizados con 1 solo comando.

2 Comparación de Despliegue: E-Commerce Gamer vs JobFlow

En tu dashboard de Render tienes actualmente dos proyectos con estrategias de despliegue y tecnologías completamente distintas. Comprender esta diferencia es clave para tu formación profesional:

1. ¿Cómo está desplegado JobFlow? (Despliegue Nativo con Buildpack de Node)

JobFlow es una API desarrollada en Node.js / Express. En Render seleccionaste el runtime "Node".

- **Cómo funciona:** Render provee una máquina virtual estándar con Node y npm preinstalados. Render ejecuta "npm install" y luego "npm start" directamente sobre su sistema operativo.
- **Ventaja:** Es muy rápido y simple de configurar para aplicaciones puramente en JavaScript/TypeScript.
- **Limitación:** Estás atado a las versiones de Node y librerías del sistema que Render ofrece. Si tu proyecto fuera en Java o necesitara Nginx, el runtime de Node no sirve.

2. ¿Cómo está desplegado el E-Commerce Gamer? (Despliegue con Docker / Contenedores)

El E-Commerce utiliza el runtime "Docker" en Render para el Backend y "Vercel CDN" para el Frontend.

- **Cómo funciona:** Render no intenta adivinar el lenguaje ni usar sus paquetes nativos. Simplemente toma tu "Dockerfile", descarga la imagen oficial de Java 21 Alpine, compila el código adentro y levanta el contenedor aislado.
- **Ventaja:** Independencia absoluta. El mismo Dockerfile corre en Render, en tu PC local con Docker Desktop, en AWS, en Azure o en Google Cloud sin cambiar una sola coma.

Cuadro Comparativo Detallado

Criterio	JobFlow (Deploy Nativo Node)	E-Commerce Gamer (Deploy con Docker)
Tecnología	Node.js / Express (Monolenguaje)	Java 21 Spring Boot + React 18 + Nginx (Políglota)
Entorno de Ejecución	Entorno administrado por Render (Buildpack)	Contenedor Linux Alpine personalizado e inmutable
Control de Versiones	Limitado a las versiones de Node de Render	Control total: Java 21, Maven 3.9, Node 20, Nginx Alpine
Portabilidad Cloud	Acoplado a plataformas que soporten Node	100% portable a cualquier nube (AWS, GCP, Azure, etc.)
Ejecución Local	Requiere Node instalado en la máquina local	Se levanta con "docker compose up" sin instalar Java ni Node
Aislamiento y Seguridad	Comparte dependencias del sistema anfitrión	Aislado en sandbox con usuario no-root por seguridad
Optimización de Peso	Sube la carpeta con todas las dependencias	Multi-stage: Descarta compiladores y reduce tamaño

3 Arquitectura Final de Producción en la Nube (Cloud Topology)

Para lograr la máxima velocidad y disponibilidad sin costo, separamos las capas de la aplicación aprovechando lo mejor de cada plataforma:

Capa	Plataforma	URL / Acceso	Rol en la Arquitectura
Frontend (UI)	Vercel (Edge CDN)	https://...vercel.app	Distribución global de la SPA, carga instantánea y SSL.
Backend (API)	Render (Docker Web Service)	https://...onrender.com/api	Servidor Spring Boot 3 con Java 21 en contenedor Docker.
Base de Datos	MongoDB Atlas (AWS Cloud)	Cluster MongoDB en la nube	Persistencia permanente NoSQL con réplicas y alta disponibilidad.

4 Código Clave Implementado para la Contenedorización

1. Multi-Stage Build del Backend (backend/Dockerfile):

BACKEND/DOCKERFILE

```
# ETAPA 1: BUILD CON MAVEN Y JAVA 21
FROM maven:3.9.6-eclipse-temurin-21-alpine AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests

# ETAPA 2: RUNTIME ULTRA LIVIANO CON JRE 21
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

2. Servidor Nginx para React SPA (frontend/nginx.conf):

FRONTEND/NGINX.CONF

```
server {
    listen 80;
    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html; # Soporte React Router
    }
}
```

3. Orquestador Local (docker-compose.yml):

DOCKER-COMPOSE.YML

```
services:
  backend:
    build: ./backend
    ports: ["8080:8080"]
    environment:
      - SPRING_DATA_MONGODB_URI=mongodb+srv://... (Atlas)
  frontend:
    build: ./frontend
    ports: ["5176:80"]
    depends_on: [backend]
```

5 Conclusión Académica y Profesional

“ Síntesis para tu Examen o Presentación ”

- JobFlow utiliza un despliegue nativo rápido ideal para APIs monolíticas en Node.js.
- E-Commerce Gamer implementa una arquitectura moderna basada en Contenedores Docker y Microservicios, permitiendo desacoplar Java 21, React y Nginx de forma estandarizada y lista para escalar en la nube.
- Ambas soluciones son válidas, pero Docker representa el estándar supremo de la industria en DevOps y arquitecturas Cloud Native.